

Module 8: Trees

Part 3: Boosting

Wenting Xu

Iowa State University

Copyright and Attribution

Some lecture slides and instructional materials in this course are adapted from the following sources:

- *An Introduction to Statistical Learning: With Applications in R (Second Edition)*

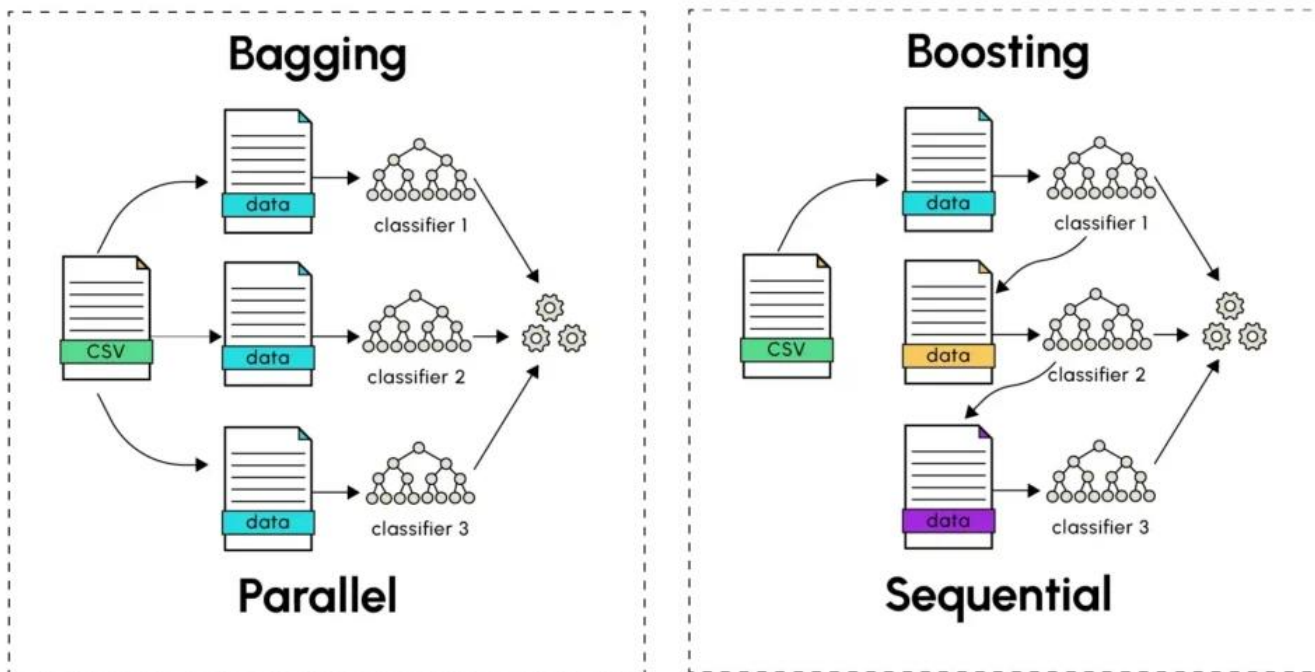
Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani
Springer, 2021

- Online course materials developed by Trevor Hastie, Robert Tibshirani, and collaborators.

- <https://liora.io/en/bagging-vs-boosting>

Boosting

- Another ensemble method. Aggregating many trees.
- Boosting does not involve bootstrap sampling.
- **Each tree is grown sequentially:** each tree is grown using information from previously grown trees.



Boosting

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.

$y_i = y - \hat{y} = y$

2. For $b = 1, 2, \dots, B$, repeat: $\hookrightarrow$ residuals
 - a. Fit a tree $\hat{f}^b$ with d splits ($d + 1$ terminal nodes) to the training data: (X, r) . **Note: We fit the tree using the current residuals, rather than Y , as the response**
 - b. Update $\hat{f}$ by adding in a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

- c. Update the residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

3. Output the boosted model,

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x).$$

Boosting

2. For $b = 1, 2, \dots, B$, repeat:

a. Fit a tree $\hat{f}^b$ with d splits ($d + 1$ terminal nodes) to the training data: (X, r) . **Note: We fit the tree using the current residuals, rather than Y , as the response**

b. Update $\hat{f}$ by adding in a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

c. Update the residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

3. Output the boosted model,

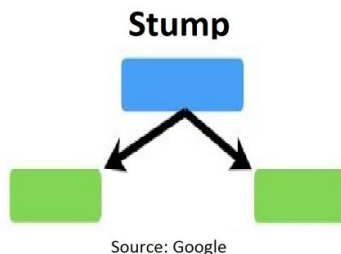
$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x).$$

Boosting

- Boosting approach learns slowly.
- Each tree is a 'weak learner' and the next tree is sequentially improved based on information from the current tree.
- How do we improve each tree? We fit the tree using **the current residuals, rather than Y , as the response.**
- We then add this new tree to the previous tree in order to update the residuals.
- Each of the individual trees can be small (hence the slow learning):
- By fitting small trees to the residuals, we slowly improve $\hat{f}$ in the areas where it does not perform well.

Boosting

1. **The number of trees B .** Unlike bagging and random forests, boosting can overfit if B is too large.
2. **Shrinking parameter λ ,** a small positive number. Typical values are 0.01 or 0.001, but the right choice can depend on the problem.
3. The number of d splits in each tree, which controls the complexity of the boosted ensemble. Often $d = 1$ works well, in which case each tree is a **stump**, consisting of a single split. More generally d is known as the interaction depth.



Boosting

- Tuning these parameters is more of an art than science. In practice, we do a grid search combined with cross-validation.
- The value for d is usually less important and we pick something that will lead to a **shallow tree**.

Implementation

See R script: boosting.R